BYTEHUM: FAST AND ACCURATE QUERY-BY-HUMMING IN THE WILD

Xingjian Du, Pei Zou, Mingyu Liu, Xia Liang, Minghang Chu, Bilei Zhu

ByteDance AI Lab

ABSTRACT

Query by Humming (QBH) is a practically meaningful task, while most existing methods struggle to scale to real-life applications due to the complex preprocessing for building the database and the limited search speed. In this paper, we propose the ByteHum system, a fast and efficient humming retrieval system which is capable of searching against large-scale databases built on raw song audios without the need for extensive preprocessing. ByteHum employs a convolutional neural network to extract features from raw audio, and utilizes a source-separated cover song identification dataset for weakly supervised training of the feature extractor. We explore the use of unsupervised domain adaptation techniques to enhance the performance of our weakly supervised model on the QBH task. Furthermore, to evaluate QBH systems' performance on non-manually processed databases in the wild, we annotate original recordings for three existing QBH benchmark sets. Our experimental results demonstrate that ByteHum significantly outperforms existing QBH systems in terms of speed and accuracy under both classical and unconstrained settings.

Index Terms— Query by Humming, Music Intormation Retrieval, Cover Detection, ByteHum

1. INTRODUCTION

Query by Humming (QBH) is a specialized Music Information Retrieval (MIR) task that identifies musical compositions from vocal inputs such as humming or singing. Traditional QBH systems pre-construct a reference database using features extracted from a curated collection of musical pieces. When a vocal query is received, the system performs a feature-based comparative analysis against this database to identify the closest musical match.

Over the past three decades, QBH has seen a mass of algorithmic approaches, the majority of which prioritize pitch contour as the key feature for musical retrieval [1–9]. These algorithms diverge mainly in their methods for pitch extraction and subsequent matching. For example, Ghias *et al.* [1] utilized autocorrelation, maximum likelihood and cepstrum analysis for pitch tracking, and employed an approximate string matching method for database searching. Wang *et al.* [2] proposed to use autocorrelation for fundamental frequency (F0) esimation, and incorprated note-based Earth Mover Distance (EMD) and frame-based Dynamic Time Warping (DTW) for pitch matching. Salamon *et al.* [4] utilized a salience-based pitch tracker, coupled with a dynamic programing method, i.e., Q_{max}, to measure the similarity between pitch contours. More recently, Mostafa *et al.* [5] combined Hidden Markov Model (HMM) and Convolutional Neural Network (CNN) for humming transcription and used advanced indexing techniques like Locality Sensitive Hashing to improve search efficiency.

Despite extensive research, current QBH systems face several challenges in real-world applications. First, the databases often rely on human-labeled MIDI files, which are costly and time-consuming to produce. Although there has been attempts [3, 4, 10] to use automatic extraction features, results indicate that these systems struggle to perform as well as MIDI-based systems. Second, traditional QBH systems use time-series alignment algorithms like Earth Mover's Distance and Dynamic Time Warping, which are computationally expensive. Although advanced indexing techniques like Locality Sensitive Hashing have been tried to speed up feature retrieval, this usually comes at the cost of lowering the identification hit rate [3]. Third, these time-series alignment techniques are sensitive to the starting position of the query, limiting their effectiveness in real-world scenarios where a query might start at an arbitrary position in the song.

On the other hand, Cover Song Identification (CSI), which is a congeneric task to QBH, has made great progress in recent years, mostly owing to the use of advanced deep learning and metric learning techniques. Our previous work in the ByteCover series [11–13] has achieved state-of-the-art performance on mainstream CSI datasets. In [4], Salamon *et al.* proposed to consider QBH as a special case of CSI. This motivates us to leverage these advancements in CSI research to improve QBH systems.

We introduce ByteHum, an extension of ByteCover3, designed to address the challenges faced by QBH. ByteHum differs from existing QBH systems in several key aspects. It learns QBH features directly from data, eliminating the need for separate pitch detection methods. This results in more discriminative and robust features, leading to better QBH performance. Additionally, ByteHum's database is composed of these learned features, removing the need for MIDI files and significantly reducing the cost of building a QBH system. The features extracted by ByteHum have a fixed dimension, allow-

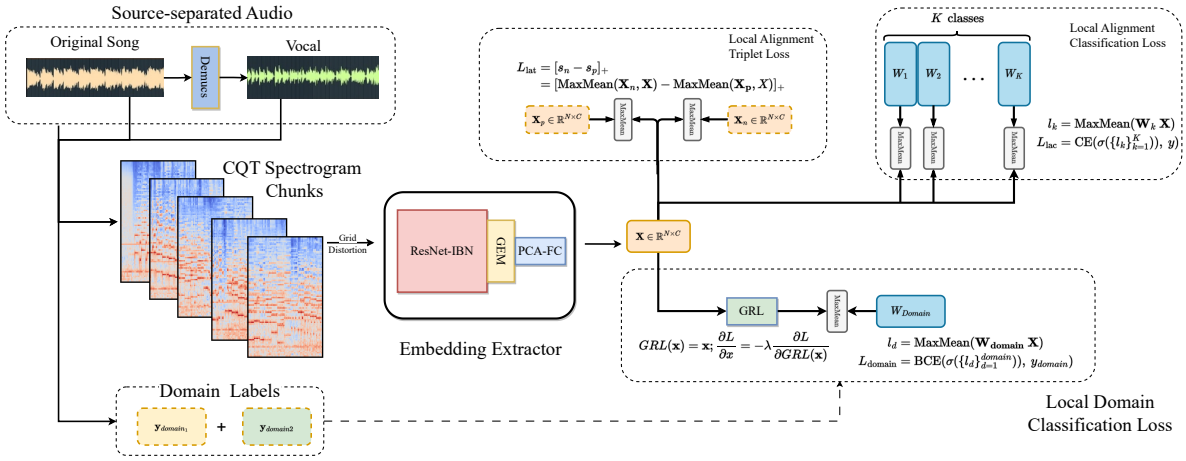


Fig. 1. Schematic overview of ByteHum. ByteHum takes CQT spectrograms as input and employs ResNet-IBN for feature extraction. Source-separated audios of a CSI dataset are used in ByteHum's weakly supervised training, augmented with spectrogram grid distortion. Multi-loss learning strategies, including Local Alignment Loss (LAL), are utilized for robustness. ByteHum is fine-tuned for the QBH task using domain adaptation techniques.

ing us to use advanced Approximate Nearest Neighbor search techniques to extend the size of our QBH database to an industrial scale. Finally, ByteHum employs a MaxMean function for similarity measurement between features, making it insensitive to the starting position of the query.

Our contributions are manifold. First, we propose a weakly supervised training methodology that leverages existing large-scale CSI datasets to make a QBH model both robust and generalizable. Second, we introduce a Spectrogram Grid Distortion Augmentation technique aimed at enhancing feature extraction capabilities. Finally, we incorporate Domain Adversarial Training (DAT) to ensure that the model performs consistently well across different settings.

2. BYTEHUM

In this section, we delineate the architecture and training paradigms of ByteHum, emphasizing its novelties: weakly supervised training, spectrogram grid distortion augmentation, and domain adversarial training for enhanced generalization in QBH tasks. The overarching architecture is depicted in Fig. 1.

2.1. Overall Framework of ByteHum

The architecture of ByteHum is fundamentally inspired by our previous work, ByteCover3, which utilized ResNet-IBN as the backbone model and combined softmax loss and triplet loss for multi-loss learning. ByteHum inherits the multi-loss learning paradigm from ByteCover3, with a particular focus on the utilization of local features.

In ByteHum, local features are extracted by first resampling the audio to $22,050$ Hz and dividing it into N overlapping chunks of 10 seconds each with a 3-second hop. These chunks are then transformed into Constant-Q Transform (CQT) spectrograms, which serve as the input to a ResNet-IBN model [11]. The model outputs a 4-D embedding, which is further processed by GeM pooling to obtain a compacted final embedding $\mathbf{X} \in \mathbb{R}^{N\times 512}$, containing N local features.

One of the major contributions of ByteCover3 is the Local Alignment Loss (LAL), designed to measure the similarity between sequences of vectors containing different numbers of local features. The LAL is composed of a classification loss $\mathcal{L}_{\text{lac}}$ and a triplet loss $\mathcal{L}_{\text{lat}}$. Specifically, the classification loss $\mathcal{L}_{\text{lac}}$ is defined as:

$$\mathcal{L}_{\text{lac}} = \text{CE}(\sigma(\{\text{logit}_k\}_{k=1}^{K}), y), \tag{1}$$

$$\text{logit}_k = \text{MaxMean}(\mathbf{X}, \mathbf{W}_k), \tag{2}$$

where $\text{CE}(\cdot,\cdot)$ is the cross entropy, $\sigma(\cdot)$ is the softmax function, $\mathbf{X} \in \mathbb{R}^{N\times C}$ is the final embedding with N local features extracted by ResNet-IBN, $\mathbf{W} \in \mathbb{R}^{K\times L\times C}$ is a trainable weight matrix in the linear layer before softmax and $\mathbf{W}_k \in \mathbb{R}^{L\times C}$ denotes the proxy representation for class k. *MaxMean* is a metric that is designed to calculate the similarity between two sequences of vector which takes account of the local alignment. And the triplet loss $\mathcal{L}_{\text{lat}}$ is defined as:

$$\mathcal{L}_{\text{lat}} = [\text{MaxMean}(\mathbf{X}_n, \mathbf{X}) - \text{MaxMean}(\mathbf{X}_p, \mathbf{X})]_+. \tag{3}$$

where $\mathbf{X}$, $\mathbf{X}_p$, and $\mathbf{X}_n$ are the local embeddings of anchor, positive, and negative samples. These two losses provide a robust optimization objective during training. ByteHum adopts these core components but extends them in various ways to improve performance and adaptability for new challenges in music information retrieval. The following sections will elaborate on these extensions and how they contribute to the overall effectiveness of ByteHum.

2.2. Weakly supervised training of QBH

To address the scarcity of datasets specifically designed for Query-by-Humming (QBH) tasks, we propose a novel methodology that leverages existing large-scale Cover Song Identification (CSI) datasets. Although CSI and QBH tasks are similar in that they both involve music retrieval, the nature of the queries differs. In QBH, queries are primarily human humming, whereas CSI datasets, such as SHS100K, consist mainly of professionally generated and mixed music. Our experiments confirm that models trained solely on CSI data perform suboptimally for QBH tasks. To mitigate this, we

introduce a two-fold strategy. First, we employ music source separation techniques to extract vocal tracks from mixed audio in CSI datasets, treating them as weakly labeled humming data. Specifically, we utilize the 2stem model from the Demucs framework as our separator. The extracted vocal tracks are then combined with the original CSI dataset to create a more representative training set for QBH tasks.

However, a significant distributional gap still exists between these synthetic humming data and real-world humming queries. This is primarily due to the imperfections and inaccuracies in user-hummed recordings, which introduce noise and fluctuations in melody contours. To bridge the distributional gap between the extracted vocals and genuine humming data, we propose a data augmentation technique directly applied to the compressed Constant Q Transform (CQT) spectrogram, bypassing the need to access the original waveform. Inspired by [14], we implement grid distortion as our augmentation strategy. This involves random scaling transformations in both the frequency (F) and time (T) dimensions of the spectrogram, controlled by scaling factors sx and sy. The scaling factors are bounded within ranges $[0.5, 1.5]$ in our experiments, allowing us to simulate the noise and fluctuations commonly found in user-hummed queries.

2.3. Domain Adversarial Training for Better Robustness and Generalization

As mentioned before, we employ a dataset comprising synthetic vocal data and original mixed music recordings to train our Query-by-Humming (QBH) model. From the perspective of transfer learning, the source domain, represented by our training dataset, and the target domain, represented by user-generated humming queries, are inherently different. This discrepancy motivates the use of domain adaptation techniques to bridge the gap between these domains. Specifically, we employ DAT [15] to enhance the model's domain generalization capabilities. The motivation of DAT is train the model adversarially with domain labels to encourge the model learn a domain-invariant features. A typical problem while applying the DAT is how to determine the belonging domain of every samples in the training set. Luckily, there are intrinsically two separated domains in ByteHum's training set: one is the original recordings and the other is the separated vocals. Thus we can assign a domain label $y_{domain} \in \{0, 1\}$ to every training sample with a embedding $\mathbf{X}$ generated by the ResNet-IBN feature extractor.

To implement DAT, we introduce a Gradient Reversal Layer (GRL) immediately following the feature extraction network. A domain classifier is appended after the GRL to distinguish between the synthetic vocal domain and the mixed music domain. During the forward pass, the GRL acts as an identity function. During the backpropagation phase, it reverses the sign of the gradient. Mathematically, if x is the input and L denotes the loss function, then:

$$GRL(x) = x; \quad \frac{\partial L}{\partial x} = -\lambda \frac{\partial L}{\partial GRL(x)}$$

where λ is a hyperparameter that controls the degree of gradient reversal and is configured as 0.2 in ByteHum. In the context of DAT, the L is typically a domain classfication loss L_{domain} calculated on the output of domain classifier. The reversed gradient serves as an adversarial signal that forces the feature extractor to learn a representation where the source and target domains are indistinguishable, thus achieving domain invariance. This is akin to a minimax game where the feature extractor aims to minimize $\mathcal{L}_{\text{task}}$ while maximizing $\mathcal{L}_{\text{domain}}$, leading to a domain-agnostic feature space. In paractical, we define the domain classification loss as below:

$$\mathcal{L}_{\text{domain}} = \text{BCE}(\sigma(\text{MaxMean}(\mathbf{X}, \mathbf{W}_{\text{domain}})), y_{\text{domain}}) \quad (4)$$

where $\text{BCE}(\cdot, \cdot)$ is the binary cross entropy and $\mathbf{W}_{\text{domain}} \in \mathbb{R}^{L \times C}$ denotes the weights of domain classifier. The overall loss function of our proposed model can be expressed as:

$$\mathcal{L}_{overall} = \mathcal{L}_{\text{lac}} + \mathcal{L}_{\text{lat}} + \mathcal{L}_{\text{domain}} \quad (5)$$

3. EXPERIMENTS

We utilized the SHS100K dataset [16], an extensive collection derived from the Second Hand Songs database, for the training of ByteHum. The state-of-the-art muisc source separation model, Demucs [17], was adopted for vocal separation. This resulted in a collection of 74,279 restored vocal samples. The restored vocal samples were then amalgamated with their corresponding original audio samples to form our comprehensive training dataset.

ByteHum was evaluated on three publicly available datasets: MIR-QBSH, IOACAS, and MTG-QBH. The MIR-QBSH dataset, sourced from Roger Jang's MIREX corpus[1], contains 4431 humming queries and 48 reference MIDI files. The queries are 8 seconds in length and start from the beginning of their corresponding MIDI files. The IOACAS dataset serves as a supplement to the MIR-QBSH, in which queries can be sung from any position within the MIDI[2]. We use 191 MIDI files with song title information and 404 corresponding queries in this dataset for our experiments. The MTG-QBH dataset proposed by Salamon*et al.* [4], aiming to facilitate the development of audio-to-audio QBH algorithms, contains 118 humming queries and meta-data of 481 canonical reference songs, without reference MIDI files provided[3].

During training of ByteHum, we used the weights of the pre-trained ByteCover3 model to initialize the ResNet-IBN module. Similar to ByteCover3, we implemented ByteHum in Pytorch framework and trained it using the Adam Optimizer. The learning rate and the batch size were set to 0.00001 and 64, respectively. The original audio samples and the synthetic vocal samples were mixed in a 1:1 ratio within each batch. For the testing, we used the metrics of Mean Reciprocal Rank(MRR) and top-X hit rate. We compared ByteHum

[1] https://www.music-ir.org/mirex/wiki/2020:Query_by_Singing/Humming

[2] https://music-ir.org/evaluation/MIREX/data/qbsh/IOACAS_QBH_Corpus.tar.gz

[3] https://www.upf.edu/web/mtg/mtg-qbh

Model	MRR ↑	Top-X hit rate ↑		
		1	5	10
	MIR-QBSH			
Ulfi *et al.* [8]†	0.33	0.17	0.48	0.73
Triastanto *et al.* [7]†	0.3	0.15	0.43	0.67
Mostafa *et al.* [5]†	0.92	-	-	-
ByteCover3 [13]	0.70	0.60	0.79	0.85
ByteHum-base	**0.94**	**0.90**	**0.98**	**0.99**
ByteHum-da	0.93	0.89	0.97	0.98
	IOACAS			
Ulfi *et al.* [8]†	0.20	0.11	0.27	0.38
Triastanto *et al.* [7]†	0.11	0.04	0.15	0.22
ByteCover3 [13]	0.73	0.66	0.79	0.83
ByteHum-base	**0.90**	**0.87**	**0.92**	**0.94**
ByteHum-da	0.89	**0.87**	0.91	0.93
	MTG-QBH			
Salamon *et al.* [4]	0.45	0.41	0.47	0.52
Alfaro-Paredes *et al.* [6]	0.26	0.16	0.35	0.50
ByteCover3 [13]	0.65	0.61	0.69	0.75
ByteHum-base	**0.85**	**0.81**	**0.89**	0.92
ByteHum-da	0.03	0.78	**0.89**	**0.94**

Table 1. Performance of different models on three datasets (- means that results are not shown in original papers, † means that the model is tested on reference set of annotated MIDIs).

to other QBH models on the three dataset mentioned before, with reference set built on original song recordings collected from the Internet instead of the manually annotated melody contour or MIDI files.

Table 1 displays our evaluation results on three datasets. Difference between *ByteHum-base* and *ByteHum-da* lies in that whether domain adaption training is employed. Notice that ByteHum uses non-manually processed song recorings as the reference sets instead of provided MIDI files. This is actually a harder setting compared with querying on reference MIDI files annotated manually, especially on MIR-QBSH where queries are aligned with the MIDIs. In spite of this, *ByteHum-base* and *ByteHum-da* perform better than all other systems on all three datasets, which is a strong demonstration of ByteHum's robustness and generalization ability.

To further explore the performence of our model under an unconstrained setting in the wild, we conducted another experiment on MTG-QBH. This setting aimed to test the models in a scenario more similar to real-life applications, under the following criteria: (1) Queries and references were not manually aligned; (2) Reference set were constructed using audio only without any additional manual annotation; (3) Noise songs irrelevant to the query set were included in the reference set. We adopted a inhouse collection of 100 thousand songs as a background set, and added noise to the reference set with songs sampled from the background set (all songs from the background set were treated as not matching any query). Fig. 2 illustrates the MRR results of ByteHum-base and ByteHum-da on reference sets with different amount of noise songs. Performence. Noise songs represent irrelevant songs added to the reference set as distractors. As discussed in previous researches [18, 19], an increase in the size of reference set usually results in a worse retrieval perfermance. Results

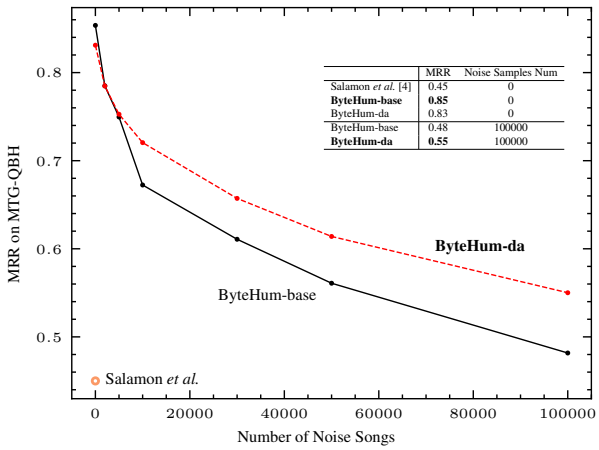


Fig. 2. For the evaluation of QBH systems in real-world scenarios, we introduce irrelevant tracks that are not part of the original candidate sets from the MTG-QBH database. The line graph depicts how the Mean Reciprocal Rank (MRR) of the music retrieval service changes as the number of these irrelevant tracks in the candidate set increases. The x-axis shows the quantity of irrelevant tracks added, while the y-axis represents the MRR on MTG-QBH.

indicate that *ByteHum-da* performs significantly better than *ByteHum-base* when the reference set gets larger, which obviously proves that the domain adaption training plays an important rule in improving the robustness of the extracted features. The experiment also demonstrates the potential of our proposed method for real-world applications, as *ByteHum-da* keeps a better MRR results than the baseline even with a large amount of noise songs in the reference set.

Our last experiment evaluates the time consumption of our model. We compared the time consumption of different models on MIR-QBSH. We didn't distinguish between *ByteHum-base* and *ByteHum-da* here as they only differ during the training phase. Results in Table 2 shows that ByteHum is distinctly faster than other systems on retrieval speed, mainly due to its use of features with fixed dimension which allows the retrieval process to use efficient cosine similarity calculations. More importantly, the most advancd Approximate Nearest Neighbor(ANN) search techniques can be used to accelerate the retrieval of ByteHum on extremely large databases, allowing us to extend the size of our QBH database to industrial scale.

	Feature Extraction	Retrival	Total
DTW	1847s	5132s	6979s
Ulfi *et al.* [8]	137s	22s	159s
ByteHum	**21s**	**4s**	**25s**

Table 2. A comparation of time consumption on feature extraction and query retrieval phases respectively.

4. CONCLUSION

In this paper, we propose a weakly supervised training methodology that leverages existing large-scale CSI datasets

to make a QBH model both robust and generalizable. The combination of spectrogram grid distortion augmentation and domain adaption training enhances ByteHum's accuracy, and the feature with fixed dimension guarantee a fast retrieval speed. Experimental results show that ByteHum outperforms all benchmark models on three QBH datasets, and domain adaption training further improve ByteHum's robustness on larger databses, proving its potential for real-life applicaiton.

5. REFERENCES

[1] Asif Ghias, Jonathan Logan, David Chamberlin, and Brian C Smith, "Query by humming: Musical information retrieval in an audio database," in *Proceedings of the third ACM international conference on Multimedia*, 1995, pp. 231–236.

[2] Lei Wang, Shen Huang, Sheng Hu, Jiaen Liang, and Bo Xu, "An effective and efficient method for query by humming system based on multi-similarity measurement fusion," in *2008 International Conference on Audio, Language and Image Processing*. IEEE, 2008, pp. 471–475.

[3] Matti Ryynanen and Anssi Klapuri, "Query by humming of midi and audio using locality sensitive hashing," in *2008 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE, 2008, pp. 2249–2252.

[4] Justin Salamon, Joan Serra, and Emilia Gómez, "Tonal representations for music retrieval: from version identification to query-by-humming," *International Journal of Multimedia Information Retrieval*, vol. 2, no. 1, pp. 45–58, 2013.

[5] Naziba Mostafa and Pascale Fung, "A note based query by humming system using convolutional neural network.," in *INTERSPEECH*, 2017, pp. 3102–3106.

[6] Edwin Alfaro-Paredes, Leonardo Alfaro-Carrasco, and Willy Ugarte, "Query by humming for song identification using voice isolation," in *International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems*. Springer, 2021, pp. 323–334.

[7] Andreas Novian Dwi Triastanto and Rila Mandala, "Query by humming music information retrieval using dnn-lstm based melody extraction and noise filtration," in *2022 5th International Conference on Information and Communications Technology (ICOIACT)*. IEEE, 2022, pp. 503–508.

[8] Muhammad Ulfi and Rila Mandala, "Improving query by humming system using frequency-temporal attention network and partial query matching," in *2022 9th International Conference on Advanced Informatics: Concepts, Theory and Applications (ICAICTA)*. IEEE, 2022, pp. 1–6.

[9] Adhiraj Banerjee and Vipul Arora, "wav2tok: Deep sequence tokenizer for audio retrieval," in *The Eleventh International Conference on Learning Representations*, 2022.

[10] Alexander Duda, Andreas Nürnberger, and Sebastian Stober, "Towards query by singing/humming on audio databases.," in *ISMIR*, 2007, pp. 331–334.

[11] Xingjian Du, Zhesong Yu, Bilei Zhu, Xiaoou Chen, and Zejun Ma, "Bytecover: Cover song identification via multi-loss training," in *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2021, pp. 551–555.

[12] Xingjian Du, Ke Chen, Zijie Wang, Bilei Zhu, and Zejun Ma, "Bytecover2: Towards dimensionality reduction of latent embedding for efficient cover song identification," in *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2022, pp. 616–620.

[13] Xingjian Du, Zijie Wang, Xia Liang, Huidong Liang, Bilei Zhu, and Zejun Ma, "Bytecover3: Accurate cover song identification on short queries," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.

[14] Curtis Wigington, Seth Stewart, Brian Davis, Bill Barrett, Brian Price, and Scott Cohen, "Data augmentation for recognition of handwritten words and lines using a cnn-lstm network," in *2017 14th IAPR international conference on document analysis and recognition (ICDAR)*. IEEE, 2017, vol. 1, pp. 639–645.

[15] Yaroslav Ganin and Victor Lempitsky, "Unsupervised domain adaptation by backpropagation," in *International conference on machine learning*. PMLR, 2015, pp. 1180–1189.

[16] Xiaoshuo Xu, Xiaoou Chen, and Deshun Yang, "Key-invariant convolutional neural network toward efficient cover song identification," in *2018 IEEE International Conference on Multimedia and Expo (ICME)*. IEEE, 2018, pp. 1–6.

[17] Doyeon Kim, Soo-Whan Chung, Hyewon Han, Youna Ji, and Hong-Goo Kang, "Hd-demucs: General speech restoration with heterogeneous decoders," *arXiv preprint arXiv:2306.01411*, 2023.

[18] Joan Serra, Emilia Gómez, and Perfecto Herrera, "Audio cover song identification and similarity: background, approaches, evaluation, and beyond," *Advances in music information retrieval*, pp. 307–332, 2010.

[19] Justin Salamon and Martin Rohrmeier, "A quantitative evaluation of a two stage retrieval approach for a melodic query by example system.," in *ISMIR*, 2009, pp. 255–260.